GraphRAG-Powered Cyber Threat Intelligence Platform

A Multi-Model NoSQL Approach

Submitted by

Shivansh Sharma — 23BCE0254
Himali Sharma — 23BCE0317
Aayush Sinha — 23BCE0558

VIT Vellore
Guide: Dr. Lydia Jane G

https://github.com/Shivansharmaaaa/graphrag

September 2026 · results updated October 2026

1. Progress Report

Progress Report — GraphRAG-Powered Cyber Threat Intelligence Platform

A Multi-Model NoSQL Approach

Project title	GraphRAG-Powered Cyber Threat Intelligence Platform: A Multi-Model NoSQL Approach
Team members	Shivansh Sharma (23BCE0254), Himali Sharma (23BCE0317), Aayush Sinha (23BCE0558)
Institution	VIT Vellore
Guide	Dr. Lydia Jane G
Repository	https://github.com/Shivansharmaaaa/graphrag
Report date	2026-09-25 (results updated 2026-10-25)

1. Problem statement

Cyber threat intelligence data is inherently polyglot: relationships between actors, campaigns, malware and infrastructure are graph-shaped; reports are schema-variable documents; active indicators of compromise need millisecond key-value lookups with expiry; and analyst questions need semantic retrieval over free text. Forcing all of this into a single database — relational or otherwise — produces either a graph database straining to store full report bodies, or a document store faking multi-hop traversal with application-side joins. This project builds a **polyglot-persistence** CTI workstation that gives each workload its natural engine, and adds a GraphRAG layer that assembles evidence from all four stores before a local LLM generates a grounded, citation-checked answer.

2. Objectives

1. Ingest STIX 2.1 bundles and free-text CTI reports into all four stores consistently.
2. Answer multi-hop graph questions (shared infrastructure, shared tooling, common victims) directly from Neo4j.
3. Serve full report bodies and schema-variable fields from MongoDB, with TLP-based redaction.
4. Maintain a hot IOC blocklist in Redis with per-key TTL, used by a strict, fail-closed

enforcement API that never falls back to another store on a miss.

5. Support semantic search over report chunks in Qdrant with actor/TLP/date payload filters.

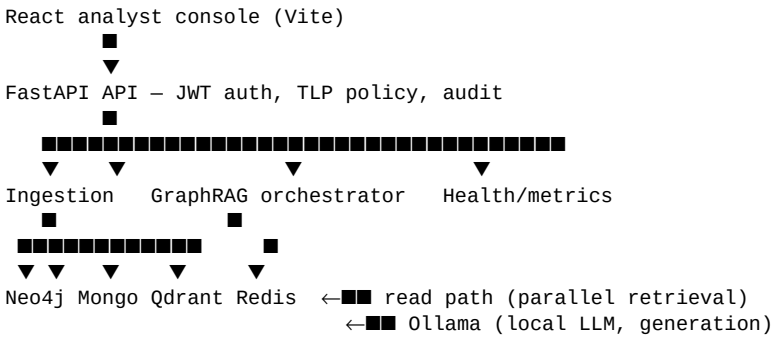
6. Orchestrate a GraphRAG pipeline: entity linking → parallel 4-channel retrieval → grounded local-LLM generation → citation validation → audit log.

7. Enforce TLP (WHITE/GREEN/AMBER/RED) access control end-to-end, including inside GraphRAG context and generated answers.

8. Ship a working analyst console (React) covering dashboard, graph explorer, report browser, semantic search, ingest, IOC blocklist, audit/replay and operations.

9. Make the whole stack reproducible via Docker Compose, with deterministic seed data and restorable database dumps.

3. Architecture implemented



This matches the architecture specified in the Review-I document exactly — no fifth catch-all database, no shortcuts between stores. The full requirement-to-code traceability table lives in `README.md` §4.

4. Technologies used

Layer	Technology
Backend	Python 3.12, FastAPI, Pydantic, Uvicorn
Graph store	Neo4j 5 (community)
Document store	MongoDB 7
Key-value store	Redis 7
Vector store	Qdrant
Embeddings	TF-IDF + truncated SVD (default, offline, 64-d) or Ollama `nomic-embed-text`
LLM	Ollama, `llama3.2:3b` by default (local, no cloud API); `llama3.1:8b` optional
Frontend	React + Vite, plain CSS, Cytoscape-style custom `GraphCanvas` component

Layer	Technology
Deployment	Docker Compose (`neo4j`, `mongo`, `redis`, `qdrant`, optional `app`/`llm` profiles)
Testing	pytest (unit), `scripts/evaluate.py` (integration + acceptance + performance)

5. Database implementation

5.1 Graph schema (Neo4j)

Node labels: `ThreatActor`, `Campaign`, `Malware`, `Infrastructure`, `Technique`, `Indicator`, `Victim`, `Report`. Relationship types: `ATTRIBUTED_TO_CAMPAIGN`, `USES` / `USES_INFRA`, `HOSTS`, `EMPLOYS`, `HAS_IOC` / `OBSERVED_AS`, `TARGETS`, `MENTIONED_IN`. Uniqueness constraints on every entity's `id` (or `(type,value)` for `Indicator`), plus range indexes on `Infrastructure.value` and `Indicator.value` — all created idempotently on startup (`stores.ensure_schema`).

5.2 Document schema (MongoDB)

Collection `reports` with the core fields from the spec (`title`, `source`, `date`, `tlp`, `confidence`, `body`, `actors`, `campaigns`, `malware`, `iocs`) plus the optional per-report-type fields (`attribution`, `mitre`, `cve`, `ics_protocols`, `wormable`, `kill_switch`, `payment_brands`, `lure_themes`, `yara_rule_name`, `build_pipeline`, `package`). Indexes on `actors`, `date`, `tlp`, `iocs`. Separate `users`, `audit_log` and `ingest_jobs` collections — no fifth database was introduced for these.

5.3 Key-value schema (Redis)

`ioc:{type}:{value}` → JSON value, `SET ... EX <ttl>`; a parallel `ioc:index` sorted set scored by expiry timestamp supports O(1) "what's about to expire" queries without a Redis `SCAN`.

5.4 Vector schema (Qdrant)

Collection `threat_chunks`, cosine distance, HNSW index. Payload: `id`, `report_id`, `title`, `source`, `date`, `date_ts`, `chunk_index`, `text`, `actors`, `campaigns`, `tlp` (the last two fields are additions beyond the spec's minimum payload, added so TLP and date filters can be native Qdrant payload filters instead of a post-filter in application code).

6. Ingestion pipeline

`backend/app/ingest.py` implements both STIX 2.1 bundle ingestion and free-text ingestion behind a single job runner (`run_job`) that fans out in a fixed order — **Neo4j → MongoDB → Qdrant → Redis** — recording per-store status in a `ingest_jobs` document (`neo4j_status`, `mongodb_status`, `redis_status`, `qdrant_status`, `errors`). Writes are idempotent (`MERGE` in Neo4j, `replace_one` in MongoDB, delete+upsert in Qdrant, `SET EX` in Redis), so retrying a partially-failed job (`POST /api/ingest/jobs/{id}/retry`) is safe. No distributed two-phase commit was implemented, per the spec's explicit instruction.

Supported STIX 2.1 object types: `attack-pattern`, `campaign`, `identity`, `indicator`, `intrusion-set`, `malware`, `relationship`, `report`, `threat-actor`, `tool`, `vulnerability`.
The original STIX JSON is preserved verbatim in MongoDB alongside the derived graph/document projection.

7. GraphRAG pipeline

`backend/app/graphrag.py` implements the full pipeline from the spec:

1. **Entity linking** — resolve actor/campaign/malware names mentioned in the question to graph node IDs.
2. **Parallel retrieval** across all four channels: Qdrant (semantic chunks), Neo4j (1–3 hop traversal, hop count configurable), MongoDB (full report hydration for cited reports), Redis (live IOC state for any mentioned indicators).
3. **Context assembly** into a single evidence bundle.
4. **Grounded generation** via the local LLM (Ollama `/api/chat`), under a system prompt that forbids uncited claims, forbids treating commodity tools (Mimikatz, Cobalt Strike) as strong attribution signals, and forbids offensive-security output.
5. **Citation validation** (`enforce_grounding`) — every cited graph edge, report ID and IOC value is checked against the live stores; uncited sentences are stripped; if coverage falls below `MIN_CITATION_COVERAGE` the pipeline **degrades to retrieve-only mode** rather than returning an unsupported answer.
6. **Audit log** — every question, its linked entities, retrieval results, generated answer, citations, model, latency and citation coverage are recorded and independently re-verifiable via `POST /api/audit/{id}/replay`.

If the LLM is unavailable, the same pipeline runs and returns graph/semantic/document/IOC evidence with generation explicitly marked unavailable (retrieve-only mode), satisfying the spec's availability requirement (NFR8).

8. API implementation

32 endpoints across `auth`, `health`, `meta`, `graph`, `reports`, `blocklist`, `enforce`, `search`, `graphrag`, `audit`, `ingest`, `admin`. Full endpoint list, parameters and curl examples: `docs/api.md`. Machine-readable spec: `docs/openapi.json` (also served live at `/openapi.json`, with interactive Swagger UI at `/docs`). The graph query API is deliberately **not** an arbitrary-Cypher endpoint — it exposes fixed, parameterized templates (`shared-infrastructure`, `shared-tooling`, `common-victims`, `path`, `node/{id}`) with configurable max-hops / max-neighbours / result-limit budgets to prevent hub-node explosion.

9. Frontend implementation

React + Vite console with pages: GraphRAG console, Graph explorer, Report browser, Semantic search, Ingest, IOC blocklist (live 5s-refreshed hot set + recently-expired panel), Audit & replay, Operations (store health, latency percentiles, integrity check). Custom components: `GraphCanvas` (force-directed graph rendering with edge-type coloring), `SimilarityMap` (Qdrant neighbourhood plot), `Ttl` (live-ticking countdown), `TlpBadge`, `Json`, `ErrorBox`.

10. Authentication / TLP

JWT-based auth (`POST /api/auth/login`, 8-hour expiry). Four demo accounts spanning the full role/clearance matrix (see §14). TLP enforcement is applied at every read surface: report list, report detail (`403` on insufficient clearance), semantic search results, GraphRAG retrieved context, and the audit log. The Redis enforcement plane (`/api/enforce/*`) is a separate, API-key-authenticated surface for downstream consumers (firewall/EDR/SIEM) that returns IOC values only — never report bodies — and fails closed (`503`, `blocked: true`) rather than guessing when Redis is unavailable.

11. Testing

- **Unit tests**: `backend/tests/test_units.py` — 15 tests, all passing, no live database

required (STIX parsing, IOC extraction/defanging, TLP policy, citation-coverage math, the citation forms the groundedness gate accepts).

- **Integration tests**: `backend/tests/test_api.py` — 15 tests that drive the real API against the four live stores: auth and roles, TLP on lists / search / GraphRAG context, publish → enforce → revoke, fail-closed enforcement when Redis is down, retrieve-only fallback when the LLM is down, audit replay, ingest fan-out, delete cascade and integrity. Each test removes what it writes, so the suite runs safely against the seeded demo stack.
- **Acceptance + performance**: `backend/scripts/evaluate.py`, run against the live seeded stack on 2026-10-25. The numbers below are from that run. `--check` exits non-zero when a correctness gate fails (gold recall, refusals, commodity caution, expiry, integrity).
- **CI**: `.github/workflows/ci.yml` runs all three tiers on every push against fresh Neo4j, MongoDB, Redis and Qdrant service containers (GraphRAG in retrieve-only mode, since CI has no LLM), plus the frontend build.

```
$ make test
15 passed, 15 deselected in 1.43s
$ make test-integration
15 passed, 15 deselected in 1.77s
$ make eval-check
all correctness gates passed
```

12. Acceptance test results (the six Appendix-A questions)

All six run against the live GraphRAG pipeline with the default `llama3.2:3b` model. Gold-edge and gold-report recall is **1.0** (every required graph fact and source report was retrieved) across all six; mean citation coverage **0.822**.

#	Question	Mode	Edge recall	Report recall	Citation coverage	Latency
1	Shared C2 infrastructure + hosted malware	llm	2/2	1/1	0.667	9.9 s
2	APT29 ↔ supply-chain ↔ SUNBURST	**retrieve-only** (LLM answer failed grounding, gate caught it)	1/1	1/1	1.000	9.1 s
3	Cobalt Strike users besides FIN7 + shared domains	llm	2/2	1/1	1.000	7.1 s

#	Question	Mode	Edge recall	Report recall	Citation coverage	Latency
4	Sandworm destructive campaigns + APT28 infra overlap	llm	2/2	1/1	1.000	6.4 s
5	Spearphishing-against-government analogue search	llm	0/0	1/1	0.667	9.0 s
6	Why Mimikatz is a weak attribution signal	llm	2/2	1/1	0.600	9.7 s

Test 2 is the clearest evidence the anti-hallucination gate works as designed: the LLM's first answer did not meet the citation-coverage threshold, so the pipeline **refused to return it** and fell back to retrieve-only evidence instead of silently shipping an unsupported claim. The commodity-hub caution required by test 6 (`commodity_caution_present: true`) is present.

The first acceptance run (2026-09-25, `llama3.1:8b`, untrimmed prompt) also reached 1.0 gold recall, with mean coverage 0.863 and latencies of 26–52 s per question; §13 explains the change.

13. Performance results

Metric	Target (NFR2)	Measured	Met?
Redis GET p99	< 5 ms	**1.43 ms** (harness probe) / 1.86 ms (515-call process sample)	Yes
Neo4j 1-hop p95	< 500 ms	**4.75 ms**	Yes
Neo4j 2-hop p95	< 500 ms	**6.42 ms**	Yes
Neo4j 3-hop p95	< 500 ms	**13.95 ms**	Yes
GraphRAG end-to-end p95	< 15 s	**9.9 s**	Yes

Environment: MacBook Pro (Apple Silicon), Ollama running `llama3.2:3b` on GPU via Metal, all four stores in local Docker containers, no external network calls.

All SLOs are met. Retrieval is a small fraction of the budget (`qdrant` p95 = 14.4 ms, `mongodb` p95 = 3.7 ms, `neo4j` p95 = 23.0 ms, `linking` p95 = 13.9 ms); the `llm` channel is almost all of it (p50 = 9.0 s, p95 = 9.9 s).

The first measurement (2026-09-25) missed this SLO at **52.3 s**. Profiling one call showed that

most of the time went to evaluating the prompt, not generating the answer: the context pack was about 5,000 tokens (60 graph facts plus 10 documents at 700 characters each), which took 17 s of a 24 s `llama3.1` call before the first output token. Two changes closed the gap:

Change	GraphRAG p95
Baseline: `llama3.1:8b`, full evidence pack in the prompt	37.5–52.3 s
Prompt budget (top 30 non-`MENTIONED_IN` facts, 6 documents × 300 chars, 12 IOC states), 320-token output cap, model kept loaded	21–27 s
Default model `llama3.2:3b`	**9.9–12.4 s**

The UI and the audit log still receive the full evidence; only the LLM's view is trimmed, and citation ids keep their full-pack numbering, so the groundedness gate is unchanged. The budget is configurable (`LLM_MAX_FACTS`, `LLM_MAX_DOCS`, `LLM_DOC_CHARS`, `LLM_MAX_IOCS`, `LLM_NUM_PREDICT`, `LLM_KEEP_ALIVE`). `OLLAMA_MODEL=llama3.1` remains available for fuller answers on faster hardware.

14. Demo accounts

user	password	role	TLP clearance	can
`analyst`	`analyst123`	analyst	AMBER	query everything up to AMBER
`junior`	`junior123`	analyst	GREEN	demonstrates TLP filtering (AMBER hidden)
`lead`	`lead123`	lead	RED	+ publish/revoke IOCs, ingest reports/STIX
`admin`	`admin123`	admin	RED	+ re-seed, rebuild vector index, delete reports

15. Problems encountered and solutions

- **Backend hang under `uvicorn --reload`.** The dev server accepted TCP connections but never finished ASGI startup, so every request — including `/api/health` and login — hung indefinitely. No crash or error in the logs, and CPU usage stayed low, which points to a stuck I/O wait during startup rather than a compute loop. `SIGTERM` didn't stop it; `SIGKILL` did.
  Fix: killed and restarted the process — the next run finished startup in under 10 seconds and has been stable since. The dev server now logs to a file instead of the terminal so a repeat hang is easier to debug.

- **GraphRAG p95 latency.** The first run measured 52.3 s against a 15 s target. Profiling traced it to prompt evaluation of a ~5,000-token context pack, not to retrieval. **Fix:** a prompt-side evidence budget, an output-token cap, a keep-alive, and `llama3.2:3b` as the default model (§13), now 9.9 s p95.
- **Grounding gate dropped correctly cited sentences.** Local models often write citations as `[G1-G4]`, `(G20)` or `[rpt-001]`, which the gate did not recognise, so it deleted sentences that did cite real evidence (in one answer, the sentence naming Sandworm's campaigns). **Fix:** the gate normalises these forms before checking each tag against the context pack. `(C2)` is deliberately excluded, since in CTI prose it means command-and-control. Unit tests cover both cases.
- **No APOC in the Neo4j container**, so `apoc.export.cypher.all` wasn't available for the database dump. **Fix:** wrote a small exporter (`scripts/export_neo4j.py`) that walks the graph via the driver and emits idempotent `MERGE` statements instead — no extra dependency, and the dump replays with plain `cypher-shell`.

16. Current limitations

- The default `llama3.2:3b` gives terser answers than `llama3.1:8b`, and in the latest run one of six answers fell back to retrieve-only through the grounding gate. `llama3.1` gives fuller answers but misses the 15 s target on this hardware (21–27 s p95).
- Embeddings default to TF-IDF+SVD (64-d, offline, deterministic) rather than a neural sentence-transformer; `EMBEDDER=ollama` with `nomic-embed-text` is supported but not the default, and was not what the acceptance run above used.
- The seed corpus is small (12 reports, 87 graph nodes) — enough to cover the six acceptance questions and demonstrate every mechanism, but not representative of production CTI volume.
- `audit_log` and `ingest_jobs` are empty in the current dump because the acceptance run's audit records and this session's ingest activity happened after the mongodump snapshot was taken; re-run `mongodump` after a live demo session to capture them.
- Redis dump is a point-in-time reference snapshot only (`database_dumps/redis/live_snapshot.json`) — TTLs are inherently time-bound by design (this is what UC4 demonstrates), so restoring it literally would restore already-expired keys. The recommended restore path re-seeds fresh TTLs instead (`scripts/restore_databases.sh seed`).

17. Future improvements

- Stream the LLM answer to the console (with the grounding gate applied on completion) so the first sentences appear sooner, and evaluate GPU-accelerated inference to run `llama3.1:8b` within the SLO.
- Add a neural embedding backend as the default once model size / first-run download time is acceptable for the target grading environment.
- Expand the seed corpus with more STIX-derived reports to stress-test hub-node budgets (`MAX_NEIGHBOURS_PER_NODE`, `HUB_DEGREE_THRESHOLD`) at larger scale.

18. Project checklist

Item	Status
Working prototype	**Completed** — all four stores + LLM healthy, login, dashboard, and all six acceptance questions verified live
Source code	**Completed** — `backend/`, `frontend/`
Docker Compose deployment	**Completed** — `docker-compose.yml`, `app`/`llm` profiles
Seed dataset	**Completed** — `backend/app/data.py`, auto-loaded on first boot
Neo4j database dump/recreation	**Completed** — `database_dumps/neo4j/graph_dump.cypher` (custom Cypher exporter, no APOC dependency)
MongoDB database dump	**Completed** — `database_dumps/mongodb/` (`mongodump`)
Redis dump/recreation	**Completed (reference)** — `database_dumps/redis/live_snapshot.json`; canonical restore path is the deterministic seed loader (TTLs are time-bound by design)
Qdrant snapshot/recreation	**Completed** — `database_dumps/qdrant/threat_chunks.snapshot` (native Qdrant snapshot API)
Restore scripts	**Completed** — `scripts/restore_databases.sh` (`seed` and `dumps` modes)
API documentation	**Completed** — `docs/api.md`, `docs/openapi.json`
OpenAPI specification	**Completed** — `docs/openapi.json`, live at `/openapi.json`
README	**Completed**
Architecture / requirement traceability	**Completed** — `README.md` §4
Database schema documentation	**Completed** — this report §5, `README.md`
GraphRAG documentation	**Completed** — this report §7
Security documentation	**Completed** — this report §10, `README.md` design notes
Testing documentation	**Completed** — this report §11–13
Progress report	**Completed** — this file
Unit tests	**Completed** — 15/15 passing

Item	Status
Integration tests	**Completed** — `tests/test_api.py` (15/15 passing against live stores) and `scripts/evaluate.py` (integrity, freshness, latency)
Continuous integration	**Completed** — `.github/workflows/ci.yml` (unit + integration + evaluation gates + frontend build)
End-to-end tests	**Completed** — full ingest→graph→document→vector→redis chain exercised by the evaluation harness and manually via the UI
Six GraphRAG acceptance tests	**Completed, functionally correct** — 1.0 gold-edge/report recall on all six; one (#2) correctly caught and demoted by the grounding gate rather than passing a bad answer
Performance measurements	**Completed** — all SLOs met, including GraphRAG p95 9.9 s against 15 s (§13)

2. API Documentation

API Documentation

Reference for the `GraphRAG CTI Platform` REST API (v1.0.0), FastAPI backend.

Interactive Swagger UI: `GET /docs`. Raw OpenAPI spec: `GET /openapi.json` (also checked in at `docs/openapi.json`).

Authentication

Most endpoints require a JWT bearer token obtained from `POST /api/auth/login`. Pass it as `Authorization: Bearer <token>`.

Enforcement endpoints (`/api/enforce/*`) instead require an API key header: `X-API-Key: edr-consumer-key`.

Demo accounts

user	password	role	TLP clearance
analyst	analyst123	analyst	AMBER
junior	junior123	analyst	GREEN
lead	lead123	lead	RED (+ publish/ingest)
admin	admin123	admin	RED (+ seed/reindex/delete)

Quick-start examples

```
# 1) login, capture the token
TOKEN=$(curl -s -X POST http://localhost:8000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"analyst","password":"analyst123"}' | python3 -c "import sys,json;print(json.load(sys.stdin)['t

# 2) system health (all four stores + LLM)
curl -s http://localhost:8000/api/health | python3 -m json.tool

# 3) graph traversal — shared C2 infrastructure
curl -s -H "Authorization: Bearer $TOKEN" \
  "http://localhost:8000/api/graph/shared-infrastructure/apt28"

# 4) semantic search
curl -s -X POST http://localhost:8000/api/search/semantic \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"query":"spearphishing against government organisations","top_k":5}'

# 5) GraphRAG question with grounded citations
curl -s -X POST http://localhost:8000/api/graphrag/ask \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"question":"Which actors share command-and-control infrastructure, and what malware was hosted there?"}'

# 6) enforcement-plane lookup (API key, not JWT — Redis only, fail-closed)
```

```
curl -s -H "X-API-Key: edr-consumer-key" \
  "http://localhost:8000/api/enforce/check?type=ip&value=185.86.77.12"
```

Endpoints

admin

`GET /api/admin/integrity`

Admin Integrity

Requires authentication.

Responses: 200

`POST /api/admin/reindex`

Admin Reindex

Requires authentication.

Responses: 200

`POST /api/admin/seed`

Admin Seed

Requires authentication.

Responses: 200

audit

`GET /api/audit`

Audit List

Requires authentication.

Responses: 200

`GET /api/audit/{audit_id}`

Audit Get

Requires authentication.

Parameters

Name	In	Required	Type	Description
`audit_id`	path	True	string	

Responses: 200, 422

`POST /api/audit/{audit_id}/replay`

Audit Replay

Requires authentication.

Parameters

Name	In	Required	Type	Description
`audit_id`	path	True	string	

Responses: 200, 422

auth

`POST /api/auth/login`

Login

Request body (`application/json`): `LoginIn`

Responses: 200, 422

`GET /api/auth/me`

Me

Requires authentication.

Responses: 200

blocklist

`GET /api/blocklist`

Blocklist List

Requires authentication.

Responses: 200

`POST /api/blocklist`

Blocklist Publish

Requires authentication.

Request body (`application/json`): `PublishIn`

Responses: 200, 422

`GET /api/blocklist/{ioc_type}/{value}`

Blocklist Get

Requires authentication.

Parameters

Name	In	Required	Type	Description
`ioc_type`	path	True	string	
`value`	path	True	string	

Responses: 200, 422

`DELETE /api/blocklist/{ioc_type}/{value}`

Blocklist Revoke

Requires authentication.

Parameters

Name	In	Required	Type	Description
`ioc_type`	path	True	string	
`value`	path	True	string	

Responses: 200, 422

enforce

`GET /api/enforce/check`

Enforce Check

Parameters

Name	In	Required	Type	Description
`type`	query	True	string	
`value`	query	True	string	
`x-api-key`	header	False		

Responses: 200, 422

`GET /api/enforce/export`

Enforce Export

Parameters

Name	In	Required	Type	Description
`format`	query	False	string	
`x-api-key`	header	False		

Responses: 200, 422

graph

`GET /api/graph/common-victims`

Q Common Victims

Requires authentication.

Parameters

Name	In	Required	Type	Description
`actor`	query	False		

Responses: 200, 422

`GET /api/graph/indicator/{value}`

UC4: historical HAS_IOC / OBSERVED_AS presence + citing documents, regardless of TTL.

Requires authentication.

Parameters

Name	In	Required	Type	Description
`value`	path	True	string	

Responses: 200, 422

`GET /api/graph/node/{node_id}`

Graph Node

Requires authentication.

Parameters

Name	In	Required	Type	Description
`node_id`	path	True	string	
`hops`	query	False	integer	

Responses: 200, 422

`GET /api/graph/overview`

Graph Overview

Requires authentication.

Parameters

Name	In	Required	Type	Description
`limit`	query	False	integer	
`include_reports`	query	False	boolean	

Responses: 200, 422

`GET /api/graph/path`

Q Path

Requires authentication.

Parameters

Name	In	Required	Type	Description
`src`	query	True	string	
`dst`	query	True	string	
`max_len`	query	False	integer	

Responses: 200, 422

`GET /api/graph/search`

Graph Search

Requires authentication.

Parameters

Name	In	Required	Type	Description
`q`	query	False	string	
`label`	query	False		
`limit`	query	False	integer	

Responses: 200, 422

`GET /api/graph/shared-infrastructure`

Q Shared Infra

Requires authentication.

Parameters

Name	In	Required	Type	Description
`actor`	query	False		

Responses: 200, 422

`GET /api/graph/shared-tooling`

Q Shared Tooling

Requires authentication.

Parameters

Name	In	Required	Type	Description
`malware`	query	False		
`actor`	query	False		

Responses: 200, 422

`GET /api/graph/tool-users/{malware_id}`

Q Tool Users

Requires authentication.

Parameters

Name	In	Required	Type	Description
`malware_id`	path	True	string	

Responses: 200, 422

graphrag

`POST /api/graphrag/ask`

Ask

Requires authentication.

Request body (`application/json`): `AskIn`

Responses: 200, 422

health

`GET /api/health`

Health

Responses: 200

ingest

`GET /api/ingest/jobs`

Ingest Jobs

Requires authentication.

Responses: 200

`POST /api/ingest/jobs/{job_id}/retry`

Ingest Retry

Requires authentication.

Parameters

Name	In	Required	Type	Description
`job_id`	path	True	string	

Responses: 200, 422

`POST /api/ingest/report`

Ingest Report

Requires authentication.

Request body (`application/json`): `ReportIn`

Responses: 200, 422

`POST /api/ingest/stix`

Ingest Stix

Requires authentication.

Request body (`application/json`): `StixIn`

Responses: 200, 422

`POST /api/ingest/stix/upload`

Ingest Stix File

Requires authentication.

Parameters

Name	In	Required	Type	Description
`publish_iocs`	query	False	boolean	

Request body (`multipart/form-data`): `Body_ingest_stix_file_api_ingest_stix_upload_post`

Responses: 200, 422

meta

`GET /api/meta`

Meta

Requires authentication.

Responses: 200

reports

`GET /api/reports`

Reports

Requires authentication.

Parameters

Name	In	Required	Type	Description
`actor`	query	False		
`tlp`	query	False		
`q`	query	False		

Responses: 200, 422

`GET /api/reports/{report_id}`

Report

Requires authentication.

Parameters

Name	In	Required	Type	Description
`report_id`	path	True	string	

Responses: 200, 422

`DELETE /api/reports/{report_id}`

Delete Report

Requires authentication.

Parameters

Name	In	Required	Type	Description
`report_id`	path	True	string	

Responses: 200, 422

search

`POST /api/search/semantic`

Semantic

Requires authentication.

Request body (`application/json`): `SearchIn`

Responses: 200, 422

3. Database Dump Summary

Database dumps

Snapshots taken 2026-09-25 from the running dev stack (seed corpus + demo IOCs).

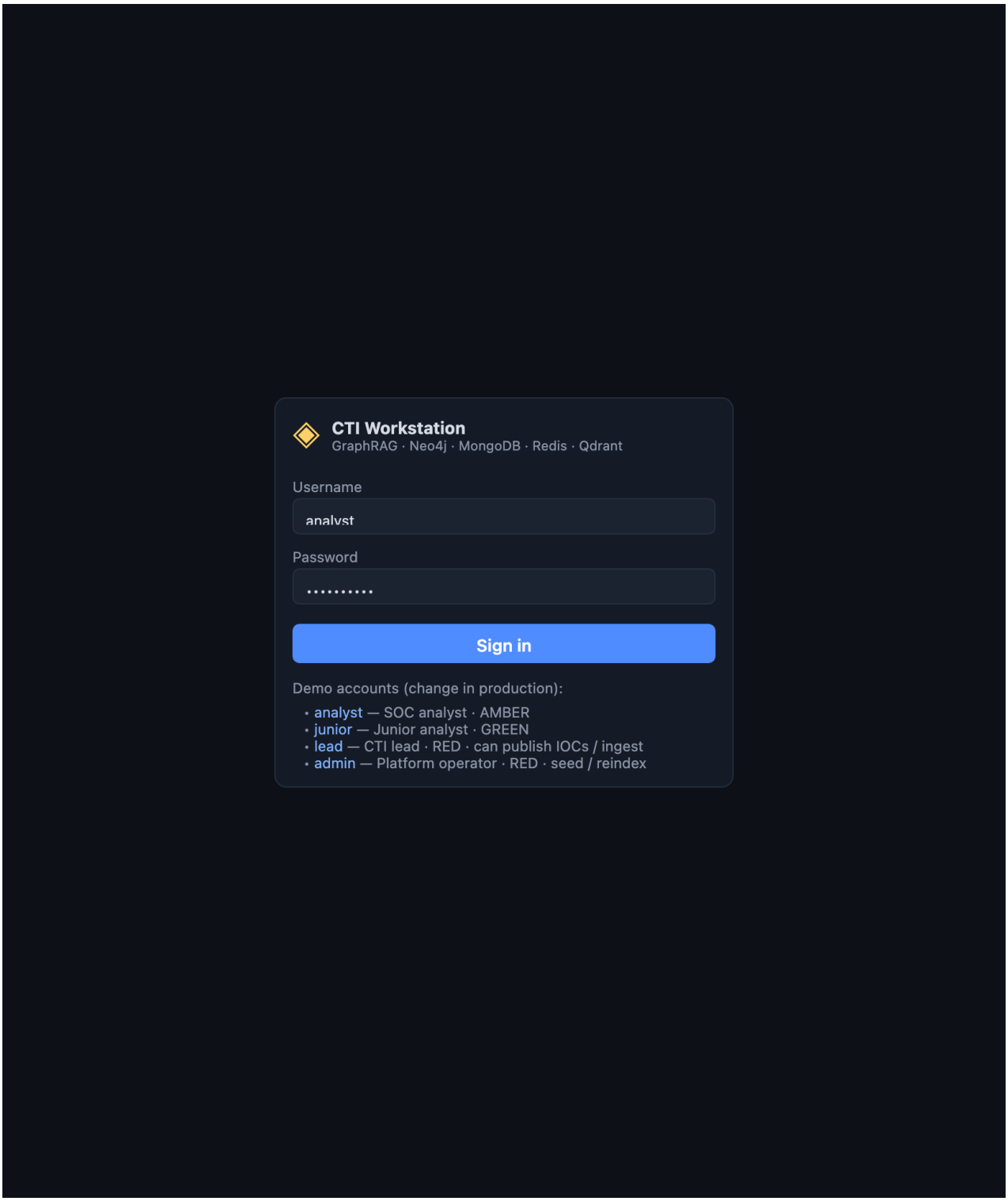
Store	File	How it was made	How to restore
MongoDB	`mongodb/cti/*.bson` + `.metadata.json`	`mongodump --uri=mongodb://localhost:27017/cti`	`mongorestore --uri=<uri> --drop database_dumps/mongodb`
Neo4j	`neo4j/graph_dump.cypher`	`scripts/export_neo4j.py` (custom Cypher export — no APOC dependency, 87 nodes / 162 relationships)	`cypher-shell -u neo4j -p <password> < database_dumps/neo4j/graph_dump.cypher`
Redis	`redis/live_snapshot.json`	`redis-cli --scan` + `GET`/`TTL` per `ioc:*` key	Reference only — IOCs are time-bound by design (FR3/UC4), so restore by re-running the seed loader instead of replaying stale TTLs
Qdrant	`qdrant/threat_chunks.snapshot`	Qdrant's native snapshot API (`POST /collections/threat_chunks/snapshots`)	`PUT /collections/threat_chunks/snapshots/recover` with `{"location": "file://qdrant/snapshots/threat_chunks.snapshot"}` (see `scripts/restore_databases.sh dumps`)

Recommended: `./scripts/restore_databases.sh seed` — rebuilds all four stores deterministically from `backend/app/data.py` (the same path the app takes on first boot). This is faster, always reproducible, and gives Redis IOCs fresh TTLs instead of already-expired ones.

`./scripts/restore_databases.sh dumps` replays the exact snapshots above instead, if you specifically need this point-in-time state.

Screenshots

Login



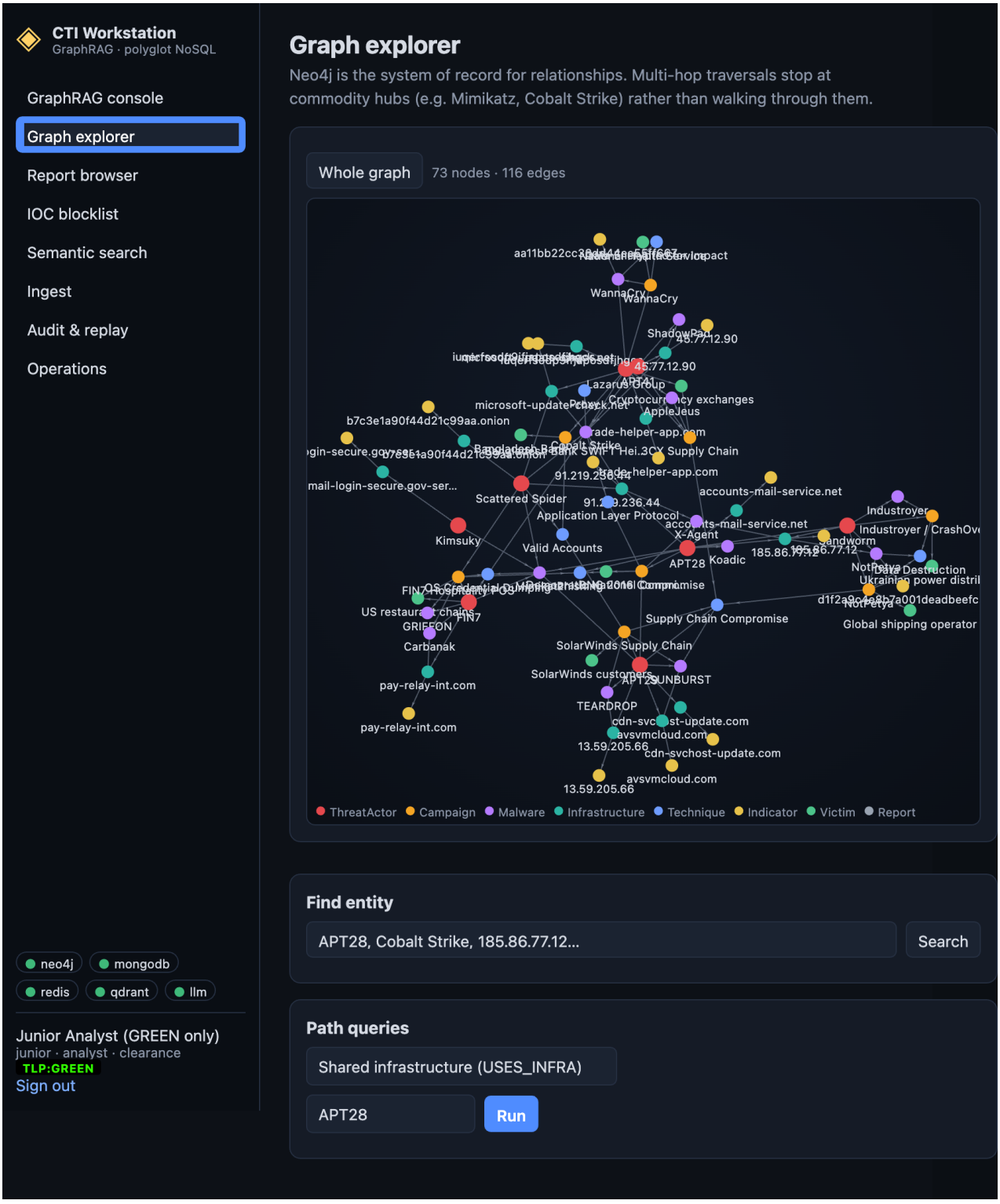
Demo-account login with TLP-clearance quick-fill buttons (analyst / junior / lead / admin).

GraphRAG console — grounded answer with citations



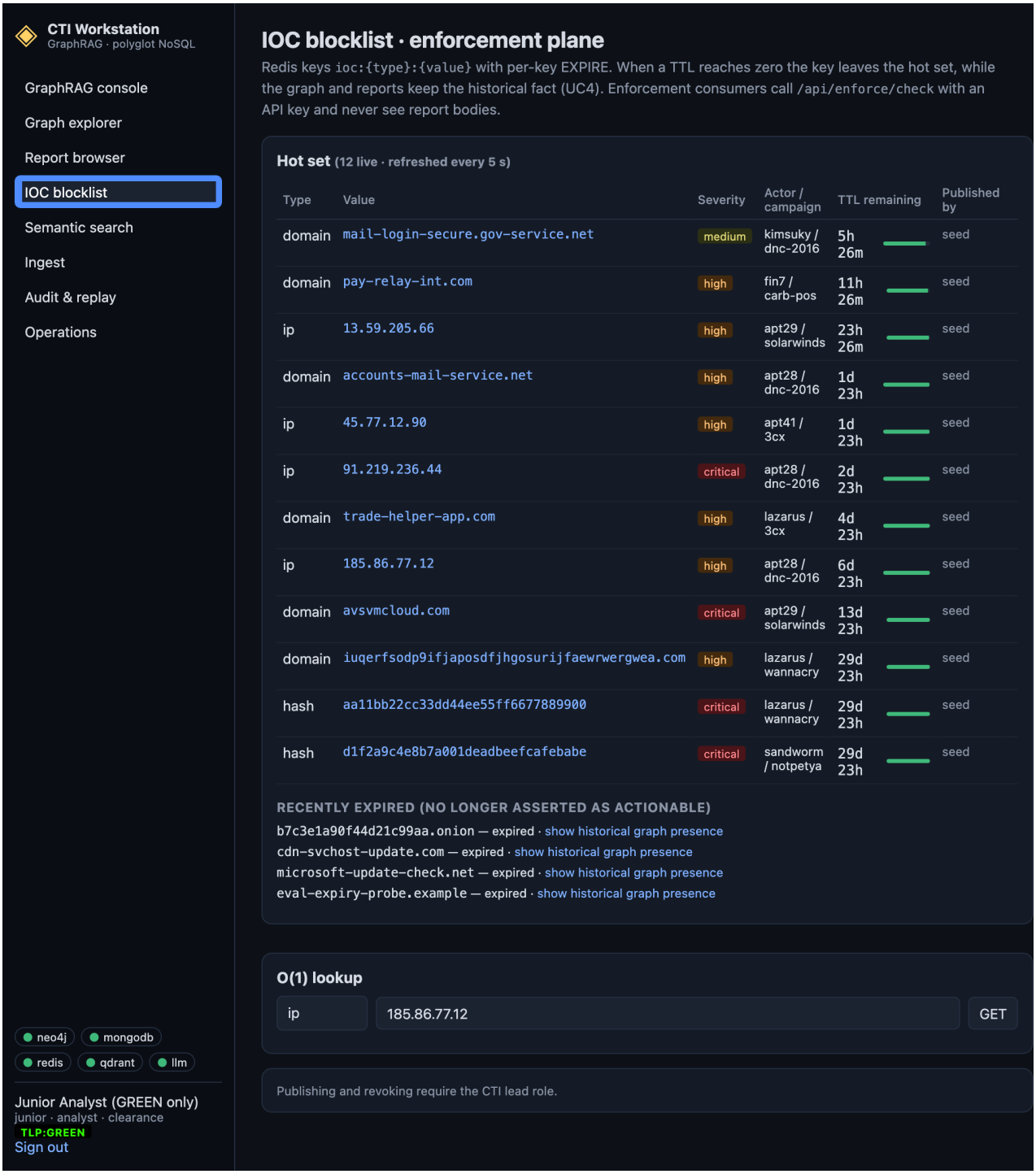
Acceptance test #6 (Mimikatz). The pipeline's grounding gate demoted this answer to retrieve-only after the LLM's first draft failed citation coverage — evidence panel shows graph edges [G#], chunks [C#], report [R:id] and live Redis IOC state [I#].

Graph explorer — whole graph



73 nodes / 116 edges: ThreatActor, Campaign, Malware, Infrastructure, Technique, Indicator, Victim and Report nodes, colour-coded by label.

IOC blocklist — Redis enforcement plane



Live hot set (12 IOCs, 5 s refresh) with per-key TTL and severity, plus recently-expired IOCs that remain historically present in Neo4j/MongoDB (UC4).